ELI V2.0 · THE COMPLETE OWNER'S MANUAL

The ELI User Manual

The full, in-depth reference to every function and feature of ELI — the architecture beneath it, the desktop app and phone dashboard, every one of the 183 things you can ask it to do, and the deep systems (memory, reasoning, voice, vision, generation, self-maintenance) that make it more than a command menu. Written for the curious beginner and the tech head alike.

Edition: v2.0.18 · **Capabilities documented:** 208 total / 183 routable · **Author:** ShadowESC95

Companions: *What ELI Is & Can Do* · *Commands & Installers* · *New User Install Guide*. If the manual and ELI ever disagree, trust ELI and file an issue — it's live software.

Contents

Part I — Understanding ELI

1 · What ELI actually is

ELI is not a chatbot bolted onto someone else's cloud API. It's a local cognitive runtime that happens to have a friendly face.

Concretely, ELI is a program that runs on *your* computer and does four things at once: it **holds a conversation** using a language model you supply, it **drives your machine** (apps, windows, files, media, shell), it **perceives** (your screen, images, documents, your voice), and it **remembers** — building a private, persistent picture of who you are and what you're working on. All of that happens without a required cloud service. You bring a local GGUF model; ELI supplies the reasoning pipeline, the agents, the memory, the senses, and the two interfaces (a desktop app and a web dashboard).

The number on the box is **208 capabilities**, of which **183 are directly routable** — meaning you can trigger them by typing or speaking. The rest are internal: synonyms that normalise to a real action, post-actions, pipeline steps, and safety guards. This manual documents all of them.

2 · The architecture, honestly

You don't need this section to use ELI. But if you want to know what's under the hood — and ELI itself will tell you if you ask — here it is without the buzzwords.

The reasoning pipeline

Every request flows through a multi-stage pipeline: **intake** (normalise the input) → **grounding** (pull relevant memory, context, and — for self-referential or generative asks — real evidence) → **routing** (decide which of the 183 actions this is, tolerant of paraphrase and typos) → **planning** (for anything non-trivial) → **execution** → **review** (a governor checks and, where needed, revises the output before you see it). A final guard discards degenerate fragments so weak model output never surfaces as a reply.

Reasoning modes & escalation

ELI has five reasoning modes — *quick*, *normal*, *advanced*, *research*, *expert* — each with its own agent time budget. It picks one based on how hard your request looks, and, crucially, it can **escalate mid-task**: if it finishes a pass and isn't confident enough, it deepens the mode and tries again. Quick mode never deepens (it's meant to be instant). Details in §26.

The agent DAG

Specialist work is done by a fleet of agents — coding, research, memory, verification, and more — run on a real **DAG orchestrator** (`eli/core/dag.py`) with parallel execution, retries, fallbacks, result caching, conditional branches, timeouts and telemetry. Ask "show your agent dag" and ELI reports the execution layers, dependencies, critical path, and last run.

Memory

Four SQLite stores (your data, agent data, a system index, and coding memory), a **FAISS vector index** for semantic recall, and a **knowledge graph** for relationships. ELI weights statements by content-derived salience, captures memorable turns on its own, and promotes facts you mention in passing into durable stores. Full detail in §25.

Senses & hands

Senses: local speech-to-text (Whisper), local text-to-speech (Piper), a local vision model for your screen and images, OCR, and optional webcam gaze. **Hands:** full OS control plus ELI's own MQTT smart-home server. **Surfaces:** a native desktop app and a FastAPI-backed web dashboard reachable from your phone on the same network.

3 · Design principles

- **Local or nothing.** Inference and memory never require the cloud. Offline is the default and it is *enforced at the network socket* — a process-wide guard, not the honour system.

- **No fake actions.** If ELI says it will do something, the pipeline re-runs and actually does it. There are no placeholder "done!" replies where nothing happened.
- **Grounded, not guessing.** When it can't verify a fact it says so. Generative work runs a real evidence step first — your code, your files, the web if you've allowed it.
- **An emergent voice.** ELI's persona is steered, not scripted. There are no canned conversational lines; it adapts to you over time.
- **Honest introspection.** Ask what it's running on, how its memory works, or why it said something, and it reports the real runtime — not a polished fiction.

Part II — Getting around

4 · First run & onboarding

The first time you launch ELI it does two things. First, it walks you through **setup and model selection** — creating its private workspace, wiring up the databases, and helping you get a model in place. Second, on a blank-slate profile it runs a short, **conversational onboarding interview**: it asks your name, what you mostly work on, how you like to be worked with (from "just the answer" to "think it through with me and ask questions back"), and what you most want ELI for.

The interview is deliberately human — each question plays off your last answer, and none of it is mandatory. Say *"skip"* at any point and ELI drops it and simply learns as you go. A substantive first message (a real task or question) also bypasses onboarding entirely. Whatever you tell it seeds your User Model, persona, knowledge graph and memory from minute one, and it keeps refining that picture forever after.

> **You need a model**
>
> A fresh install ships without the large AI "brain" file, to keep the download small. Point ELI at any local GGUF model (drop it in `models/`), or fetch the convenience pack with `./RUN_ELI.sh --with-github-assets`, or let it auto-download one sized to your VRAM. ELI tunes GPU layers, batch and context to your hardware automatically.

5 · The desktop app, tab by tab

The desktop app is home base — the full experience, with everything ELI can do. It's organised into tabs across the top.

Tab	What it's for
Chat	The heart of ELI. Talk or type; conversation, commands and answers all happen here. Most of this manual is really about what you can do from this one box.
Images	

	Image generation. A fast procedural generator is the default (instant, seed-randomised); an optional local diffusion model (SSD-1B) swaps in for true image synthesis, temporarily freeing VRAM from the language model to do it.
Proactive	ELI's initiative, in six sub-tabs — **Suggestions** (self-generated proposals), **Summaries** (session recaps), **Insights**, **Habits** (learned patterns awaiting your approval), **Self-Improve** (its own patch cycles), and **Memory**.
Quick Actions	One-click buttons for common actions, so you don't have to type them.
Screen	Screen vision — let ELI look at your display and answer questions about it, locate on-screen elements, or watch ambiently.
IDE	A built-in editor where generated scripts and scaffolded projects open, ready to run.
Eli's World	ELI's ambient, visual "inner life" — an avatar view with rooms (e.g. an Anomaly Room, a Memory Archive). Deliberately gated to world/activity questions so it never leaks fabricated activity into ordinary chat.
Labs	The workbench — eleven sub-tabs (below).
Coding	The full coding-agent surface: solve tasks, examine code, apply verified fixes, review diffs.
Tasks	Scheduled and overnight work. Create, edit and delete jobs; watch background runs. Durable across restarts, with optional nightly recurrence.
Experimental	Newer, rougher features still proving themselves.
Report Builder	Build grounded, multi-section reports through the evidence-first generation pipeline — promoted to a top-level tab because people use it a lot.
Files	Browse, read and act on your files with ELI alongside you.
Settings	Everything configurable — model & hardware, voice, network, persona, connectivity (§31).

INSIDE THE LABS WORKBENCH

Labs holds eleven tools: **Document** (grounded document authoring with a live generation plan), **Notebook**, **Memory & Conversations** (browse what ELI remembers), **Jupyter** (a real notebook surface), **Calculator**, **Physics**, **File Chat** (converse with one specific document), **Workspaces** (bundle files, folders, a memory tag and owned background tasks into a named Project), **Sim / IDE**, **Orchestration** (watch the agent DAG live), and **Test & Review** (run the suite and act on results).

6 · The web & phone dashboard

ELI is one brain with two front doors. The second is a web dashboard served by a local FastAPI server — the same ELI, in a browser, including your phone or tablet on the same Wi-Fi.

```
./scripts/eli_serve.sh --lan --https
```

- `--lan` exposes it to your home network and prints a **token-protected URL** plus a QR code (also in the app's *Connect a phone* tab). The token keeps strangers on your network out; it rides in the URL fragment so it never lands in server logs.
- `--https` adds an HTTPS port (8443) — **required for the phone microphone**, because browsers refuse to hand the mic to a plain `http://` LAN address. You'll accept a one-time self-signed-cert warning on the phone.

Everything still runs on your computer — the phone is just a window onto it. The dashboard mirrors the core surfaces (chat, devices, tasks, cognition, audit, connect) so you can drive ELI from the couch.

7 · How to talk to ELI

You never memorise commands. The router is paraphrase- and typo-tolerant, so the example phrases throughout Part III show the *shape* that fires an action, not the only wording accepted. There are three input methods, and you mix them freely.

TYPING

Type in the Chat box like a message. Plain English. "Close spotify and set an alarm for 7" is understood as readily as any exact phrase.

VOICE — PUSH TO TALK

Trigger listening and speak; ELI transcribes locally with Whisper, acts, and can reply aloud (Piper TTS) if voice output is on.

VOICE — HANDS-FREE WAKE WORD

Say the wake word ("computer, …") then your request. The wake word is yours to choose and ELI trains a local detector on it (§14, §29).

CHAINED COMMANDS

ELI parses several actions from one sentence and runs them in order: *"open spotify, play some lo-fi, and tile my windows 2x2."* This is the `MULTI_COMMAND` capability.

> **Reading Part III**
>
> Each capability lists what it does, how you'd trigger it, and — where it matters — what happens under the hood and the gotchas. `plugin` marks actions backed by a plugin; everything else is core executor + router. Voice users prefix with the wake word.

Part III — Every capability, in depth

All 183 routable actions, grouped into their twenty families. This is the exhaustive reference: if ELI can be told to do it, it's in here.

8 · Conversation & persona

Action	What it does & how to use it	Say…
`CHAT`	The open-conversation catch-all and fallback. Anything that isn't a command lands here and gets a persona-bound, grounded reply. Self-referential and relational turns go here too — never to a web search.	*"how are you", "tell me a story"*
`EXPLAIN_LAST_RESPONSE`	ELI explains *why* it gave its previous answer — the reasoning and grounding behind it.	*"why did you say that"*
`CLEAR_CHAT_HISTORY`	Clears the current conversation thread and starts clean. Durable memory is untouched.	*"start a new chat"*
`PERSONA_LOCK_SET`	Pin a role/persona for the session so ELI stays in character.	*"lock your persona to a tutor"*
`PERSONA_LOCK_CLEAR` · `PERSONA_LOCK_STATUS`	Release a pinned persona; report the current lock.	*"clear persona lock"*
`PERSONA_REFRESH`	Reload ELI's persona from your latest profile — useful after telling it a lot of new things about yourself.	*"refresh your persona"*
`HELP` · `LIST_CAPABILITIES`	List what ELI can do / enumerate its capabilities.	*"what can you do"*
`MULTI_COMMAND`	Runs several chained commands from one utterance, in order — the parser splits your sentence into discrete actions and executes each.	*"close steam and set an alarm for 7am"*

9 · App & window control

Action	What it does & how to use it	Say…
`OPEN_APP`	Launches an application. Backed by a live index of your machine's own executables, so it finds what's actually installed — and if the app is missing, it offers to install it first.	*"open spotify"*
`CLOSE_APP` · `FOCUS_APP`	Quit an app, or bring its window to the front.	*"close chrome"*
`OPEN_BROWSER` · `OPEN_URL`	Open the default browser, or go straight to a URL.	*"open github.com"*
`OPEN_IDE` · `OPEN_FILE_SYSTEM`	Open the code IDE, or the file manager.	*"open files"*
`OPEN_SYSTEM_SETTINGS`	Open OS settings.	*"open system settings"*
`OPEN_AUDIO_SETTINGS` · `OPEN_POWER_SETTINGS` · `OPEN_NETWORK_BROWSER`	Jump directly to the audio, power, or network settings panels.	*"open audio settings"*
`OPEN_COMMUNICATION_HUB` · `OPEN_MEDIA_HUB`	Open your mail/chat apps, or your media apps, as a group.	*"open media hub"*
`TILE_WINDOWS`		

	Arrange open windows in a grid — specify the layout ("2x2", "4x2").	*"tile windows 2x2"*
`MAXIMISE_WINDOW` · `MINIMISE_WINDOW` · `MINIMISE_ALL`	Maximise the focused window; minimise it; or minimise everything / show the desktop. US spellings (`MINIMIZE_*`) work too.	*"show desktop"*
`MINIMISE_APP` · `HIDE_APP`	Minimise or hide a *named* app's window specifically.	*"minimise spotify"*
`NEXT_WINDOW` · `PREVIOUS_WINDOW` · `RESTORE_WINDOWS`	Cycle between windows, or restore minimised ones.	*"next window"*
`SWITCH_WORKSPACE`	Move between virtual desktops.	*"switch to workspace 2"*
`SMART_HOME` **plugin**	Control smart-home devices through ELI's own device server (§ Smart home / §31).	*"turn off the lights"*
`CONFIRM_PENDING_REMEDIATION` · `CANCEL_PENDING_REMEDIATION`	Approve or decline a pending "install X?" / fix offer — how you answer ELI's own prompts.	*"yes" / "no"*

10 · Input control

Action	What it does	Say…
`VOLUME`	Adjust or mute system volume — relative or a specific level.	*"set volume to 30", "mute"*
`KEYBOARD`	Send keystrokes or type text into the active field.	*"press enter", "type hello world"*
`MOUSE_CONTROL`	Move and click the mouse directly.	*"left click", "move the mouse up"*

11 · Media

Action	What it does & how to use it	Say…
`PLAY_MEDIA`	Plays a specific track on the platform you name — Spotify, or YouTube audio via a headless mpv. **Explicit platform is honoured:** say Spotify and ELI won't silently fall through to YouTube; it verifies playback actually started before claiming success.	*"play Juicy by Notorious B.I.G. on Spotify"*
`PAUSE_MEDIA` · `STOP_MEDIA`	Pause or stop playback.	*"pause the music"*
`NOW_PLAYING`	Reports the current track and where it's coming from.	*"what song is this"*
`NEXT_MEDIA` · `PREVIOUS_MEDIA` · `REPEAT_MEDIA` · `SHUFFLE_MEDIA`	Full transport control.	*"skip", "shuffle"*
`MEDIA_CONTROL`	Generic transport via MPRIS for any compliant player.	*"play/ pause"*
`SKIP_YOUTUBE_AD`	Skips a YouTube ad when it becomes skippable.	*"skip the ad"*

12 · Vision & screen

Action	What it does & how to use it	Say…
`SCREEN_READ_ANALYZE`	The big one. Screenshots your display, runs it through a local vision model plus OCR, and answers questions about what's actually on screen — a real glance, not a guess.	*"what's on my screen"*
`SCREEN_LOCATE`	Finds a UI element or text on screen (pairs well with gaze or mouse control).	*"find the submit button"*
`SCREENSHOT`	Captures a screenshot to disk.	*"take a screenshot"*
`OCR_IMAGE`	Extracts text from an image file.	*"read the text in image.jpg"*
`ANALYZE_IMAGE`	Describes/analyses an image with the vision model (Qwen2.5-VL hot-swaps in; the CLIP step runs on CPU for stability).	*"describe cat.jpg"*
`ANALYZE_PDF` · `ANALYZE_PDF_FOLDER`	Reads/summarises a PDF, or every PDF in a folder.	*"summarise report.pdf"*
`ANALYZE_CSV`	Analyses a spreadsheet/CSV.	*"analyse data.csv"*
`AMBIENT_VISION`	Toggles continuous screen-watching on or off.	*"watch my screen"*

13 · Gaze (eye-tracking)

Action	What it does & how to use it	Say…
`GAZE_ENABLE` · `GAZE_DISABLE`	Turn webcam gaze control on or off.	*"enable gaze control"*
`GAZE_CALIBRATE` · `GAZE_STATUS`	Calibrate the tracker (do this once for accuracy); report status.	*"calibrate gaze"*
`GAZE_CLICK`	Voice-triggered clicks land *where your eyes are resting*. With gaze on, "open", "left click", "right click" or "hit enter" acts at your gaze point — the gaze engine supplies the coordinates and the OS controller performs the click. A genuine accessibility tool.	*"left click" (gaze on)*

14 · Voice

Action	What it does & how to use it	Say…
`DICTATE` · `TRANSCRIBE`	Dictate straight into the active field; transcribe an audio file.	*"start dictation"*
`SPEAK` **plugin**	Speak text aloud with local TTS.	*"say good morning"*
`LISTEN_FOR_COMMAND`	Start listening for a voice command; the wake word triggers it hands-free.	*"computer, …"*
`WAKE_SET`	Set your own wake word — *any* phrase. ELI persists it and trains a detector on it.	*"set my wake word to atlas"*
`WAKE_TRAIN`	Train the local, self-supervised wake-word model (Piper-synthesised samples plus noise/music augmentation, so it holds up over background audio).	*"train the wake word"*
`WAKE_ENROLL`	Record *your* voice saying the wake word and retrain, personalising detection.	*"enroll my wake word"*
`TRAIN_VOICE`	Learns how you sound — pitch, energy, rate, emotion, and question-vs-statement — then adapts ELI's own delivery to match you.	*"learn how I speak"*
`VOICE_DIAGNOSTICS`	Runs STT/TTS diagnostics.	*"run voice diagnostics"*

15 · Files & notes

Action	What it does	Say…
`CREATE_FILE` · `CREATE_FOLDER`	Create a file or folder.	*"create a file notes.txt"*
`READ_FILE` · `LIST_DIR`	Read a file's contents; list a directory.	*"list ~/Documents"*
`FILE_AUDIT` · `SUMMARIZE_FILE`	Audit files for issues; summarise a document.	*"summarise report.docx"*
`CONVERT_DOCUMENT`	Convert between document formats.	*"convert report.md to pdf"*
`WRITE_NOTE` · `NEW_NOTE` · `LIST_NOTES` · `SEARCH_NOTES` **plugin**	Jot, create, list, and search notes.	*"write a note: buy milk", "search notes for budget"*
`GET_CLIPBOARD` · `SET_CLIPBOARD`	Read from or write to the clipboard.	*"what's in my clipboard"*

16 · Generation — documents & code

Action	What it does & how to use it	Say…
`GENERATE_DOCUMENT` · `CREATE_DOCUMENT`	A grounded document via the multi-stage pipeline: evidence → plan/outline → sections → review → revise. It runs real analysis (your code/files/web) before writing, so a report about your project is factual, not generic. Single-pass fallback if the full pipeline can't run.	*"write a report on X"*
`DATA_FABRICATOR`	Generates synthetic/test data to a spec.	*"fabricate a test dataset for X"*
`GENERATE_SCRIPT` · `GENERATE_PROJECT`	Writes a runnable script, or scaffolds a whole multi-file project, via the coding agent.	*"write a bash script to monitor the GPU"*
`CODE_SOLVE`	Solves a coding task through a plan → DAG → verify → repair loop.	*"implement function X"*
`FIX_FILE` · `EXAMINE_CODE`	Find and fix bugs in a file, or run a tiered error scan (syntax → lint → gated LLM) across named files or the whole codebase.	*"fix the bugs in foo.py"*
`CONFIRM_CODE_FIX` · `CANCEL_CODE_FIX`	Apply (or decline) the offered fixes — including "just the logic ones".	*"yes, fix it"*
`SHOW_DIFF` · `OPEN_IN_IDE`	Show a code diff; open a generated artifact in the IDE (often automatic after generation).	*"show the diff"*

17 · System status & info

Action	What it does	Say…
`CPU_USAGE` · `RAM_USAGE` · `GPU_STATUS` **plugin**	Live CPU, memory, and GPU/VRAM readouts.	*"gpu status"*
`SYSTEM_STATS` · `HARDWARE_PROFILE` · `GET_STATUS`	Combined stats, the detected hardware profile, and general status.	*"system stats"*
`TIME` · `DATE` (+ `GET_TIME` / `GET_DATE`)	Current time and date.	*"what time is it"*
`GET_WEATHER` **plugin**	Weather for a place (Net-gated).	*"weather in Wexford"*
`NEWS_FETCH`	A *synthesised* news digest — never a raw dump. Blends top stories with your interests, adds a dynamic follow-up or two, and offers to go deeper (Net-gated).	*"catch me up"*
`WEB_SEARCH` **plugin**	Web search — strictly Net-gated. With Net off it refuses honestly rather than guessing (enforced inside the plugin so it can't be bypassed).	*"search the web for X"*
`MESSAGE_TIME_QUERY` · `CHECK_CHRONAL_ALIGNMENT`	"When did I say that?", plus a playful date/time sanity check.	*"when did I say that"*

18 · Grounded introspection — ask ELI about itself

Action	What it reports	Say…
`RUNTIME_STATUS`	The live model, context length, GPU layers and batch size it's actually running.	*"what are you running on"*
`RUNTIME_AUDIT` · `IMPORT_AUDIT` · `RESOLVE_RUNTIME_PATHS`	A full runtime audit, which imports are failing, and the resolved paths of critical files.	*"run a full runtime audit"*
`GUI_RUNTIME_AUDIT` · `DIAGNOSE_WRAPPERS`	Audits GUI wiring with file-read proof; shows the executor middleware/ wrapper chain.	*"prove every gui hook"*
`COGNITION_STATUS` · `EXPLAIN_COGNITION_RUNTIME`	Cognition runtime status, and a verbatim explanation.	*"cognition status"*
`MEMORY_STATUS` · `MEMORY_STATS` · `EXPLAIN_MEMORY_RUNTIME`	Memory status and counts, plus a verbatim internal explanation naming all four SQLite databases.	*"explain exactly how your memory works"*
`AWARENESS_STATUS` · `FRONTIER_STATUS`	What ELI is aware of; a full system wiring matrix.	*"what are you aware of"*
`ELI_IDENTITY_AUDIT` · `NAME_SOURCE_AUDIT`	Audits ELI's identity provenance; explains where its knowledge of your name comes from.	*"how do you know my name"*
`REASONING_MODE_STATUS` · `EXPLAIN_ALL_REASONING_MODES`	The current reasoning mode; a description of all five.	*"what reasoning mode are you in"*
`CODE_CHANGES` · `SELF_REPORT`	Recent code changes; what ELI has been working on.	*"what have you been working on"*

19 · Memory & profile

Action	What it does	Say…
`MEMORY_STORE` · `MEMORY_RECALL`	Store a durable fact; recall stored facts on a topic.	*"remember my sister's name is Anna"*
`PERSONAL_MEMORY_SUMMARY`	A summary of what ELI knows about you.	*"what do you know about me"*
`PERSONAL_MEMORY_DEEP_EXPLAIN`	A deep, *sourced* account — each fact and which store it lives in.	*"explain everything you know about me and where it's stored"*
`USER_IDENTITY_SUMMARY` · `REFRESH_USER_INFO`	Who you are; re-extract the profile from recent conversation.	*"who am I"*
`SET_USER_NAME`	Set or correct your name.	*"call me Alex"*

20 · Self-maintenance

Action	What it does	Say…
`SELF_ANALYZE` · `SELF_IMPROVE` · `SELF_PATCH`	Analyse its own failures/metrics; run a self-improvement patch cycle; apply a patch.	*"analyse your failures"*
`SELF_IMPROVEMENT_LOG`	Show the log of what it has changed about itself.	*"show your self-improvement log"*
`SELF_UPGRADE` · `SELF_UPDATE`	git pull → deps → rebuild indexes; or a control-contract self-update.	*"upgrade yourself"*
`SELF_TEST` · `RUN_TESTS` · `TEST_REVIEW`	Internal self-tests; run the full pytest suite and summarise; or run, back up the report, write an errors file, and offer result-driven fixes.	*"run the tests and tell me what to fix"*
`GENERATE_TESTS`	ELI writes and sandbox-verifies behavioural tests for its own functions.	*"grow your test coverage"*
`LORA_STATUS` · `LORA_TRAIN`	Report LoRA readiness (preflight), or run the training DAG (preflight → build → train → eval). Dry-run from chat; real training as an overnight task.	*"is lora ready"*, *"fine-tune yourself"*
`ORCHESTRATION_STATUS`	Explains the agent DAG — layers, dependencies, critical path, last run.	*"show your agent dag"*

21 · Tasks, time & planning

Action	What it does	Say…
`SCHEDULE_TASK`	Schedule overnight/ timed work — code, research, self-upgrade, reflection, eval, testgen. Durable across restarts, with optional nightly recurrence. Lands in the Tasks tab.	*"research X overnight", "build Y at 2am"*
`SET_ALARM` · `SET_TIMER`	Alarms and countdown timers.	*"set a timer for 10 minutes"*
`ADD_EVENT` · `LIST_EVENTS`	Add and list calendar events.	*"add an event tomorrow at 3pm"*
`POMODORO_START` · `POMODORO_STOP` · `POMODORO_STATUS` **plugin**	A focus timer with start/stop/status.	*"start a pomodoro"*
`BACKGROUND_JOBS` · `CHECK_JOB`	List background/ scheduled jobs; check a specific one.	*"show background jobs"*
`HABIT_STATUS` · `CONFIRM_HABIT` · `DECLINE_HABIT`	Show learned habits; approve or decline a proposed one. Habits are always proposed disabled — ELI acts only after you say yes.	*"show my habits"*
`EXECUTE_GOAL`	Execute a stored goal.	*"execute goal X"*

22 · Proactive

Action	What it does	Say…
`PROACTIVE_START` · `PROACTIVE_STOP` · `PROACTIVE_STATUS`	Start/stop/check the proactive daemon — background awareness that watches for useful moments, refreshes the self-model, and can generate goal/scheduler proposals.	*"start proactive mode"*
`GET_PROPOSALS`	ELI's self-generated proposals and goals from observed signals.	*"what's on your agenda"*
`MORNING_REPORT`	A daily briefing: news, weather, agenda in one digest.	*"give me my briefing"*

23 · Plugins

Action	What it does	Say…
`PLUGIN_LIST` · `PLUGIN_SEARCH`	List installed plugins; search for more.	*"list plugins"*
`PLUGIN_INSTALL` · `PLUGIN_UNINSTALL`	Install or remove a plugin.	*"install the X plugin"*
`PLUGIN_ENABLE` · `PLUGIN_DISABLE`	Enable or disable an installed plugin.	*"disable the X plugin"*

24 · Shell & advanced

Action	What it does	Say…
`RUN_CMD` · `SHELL_EXEC`	Run a shell command — always gated and confirmed. A risk gate and approval engine sit in front, and generated scripts pass a safety wrapper. Nothing runs silently.	*"run the command: ls -la"*
`SEQUENCE`	Run a multi-step action sequence.	*"do X then Y then Z"*
`ROUTING_FAULT_EXPLAIN`	Explains why an input failed to route — handy for learning ELI's phrasing.	*"why did that fail to route"*
`NOOP`	Internal no-op; fragmentary/degenerate input is dropped rather than acted on.	*(internal)*

The other 25

Beyond these routable actions sit internal ones — synonym normalisation (`LAUNCH_APP` → `OPEN_APP` , `WRITE_CODE` → `GENERATE_SCRIPT` , and more), post-actions, pipeline steps and guards. You never call them directly; they're what make the router forgiving and the pipeline safe.

Part IV — The deep systems

25 · Memory internals

ELI's memory is not one blob. It is **four physical SQLite databases** — a user store (conversations, memories, your profile), an agent store (agent dispatch, metrics, consolidated failures), a system index, and a coding memory — plus a **FAISS vector index** for semantic recall and a **knowledge graph** for relationships between entities.

- **Natural salience.** ELI weights statements by content-derived importance — it isn't told what matters, it infers it — and auto-captures memorable turns into durable memory.
- **Passing-fact recall.** Say something once in passing ("my dog Shadow") and it's promoted from the raw turn log into durable stores via knowledge-graph patterns and keyword fallback, so it can be recalled later — and retired facts (an old name) are suppressed.
- **Session continuity.** On session close ELI writes an LLM summary; a recent-sessions digest feeds the persona; recall is preference-weighted.
- **Transparency & control.** "What do you know about me" summarises; the deep-explain sources every fact to its database. Correct, refresh, or clear at will. It's all local.

26 · Reasoning modes & escalation

Five modes, each with its own agent time budget, selected automatically and escalated when confidence is low.

Mode	Character & when it's used
quick	Instant, direct answers and simple commands. Never deepens — it's meant to be snappy.
normal	The everyday default. Balanced reasoning for ordinary requests.
advanced	Harder problems needing more careful, multi-step reasoning.
research	Investigation across sources and agents, with a larger time budget.
expert	The deepest, most thorough pass for genuinely gnarly work.

Confidence-driven deepening: after a pass, if ELI isn't confident enough, it escalates the mode and re-runs — spending more only when the problem earns it.

27 · The generation pipeline

Anything generative — a document, a report, a self-referential explanation — runs a **hybrid evidence-planner** before writing: *plan → gather → consume*. It decides what evidence it needs, gathers it from real sources (your code via the code examiner, your files, memory, and the web if Net is on), then writes from that evidence. Documents specifically go plan/outline → sections → review → revise, so the output is grounded and structured rather than a single-shot guess. This is why a report on your own project cites your actual architecture, not a hallucinated generic one.

28 · The agent DAG

ELI's orchestrator (`eli/core/dag.py`) is a full execution engine: parallelism, retries, fallbacks, result caching, conditional branches, per-node timeouts and telemetry. Fourteen-plus agents run on it. The coding agent is the template — plan → build → verify → repair. "Orchestration status" / "show your agent dag" exposes the live execution layers, dependencies, critical path and last run.

29 · Voice stack internals

Speech-in is **Whisper** (local); speech-out is **Piper** (local). The wake word is the interesting part: you can set *any* phrase, and ELI trains a **self-supervised detector** for it — synthesising samples with Piper and augmenting with noise and music so it triggers reliably even while music plays. You can further **enroll your own voice** saying the word to personalise it. Separately, `TRAIN_VOICE` profiles how *you* sound (pitch, energy, rate, emotion, question-vs-statement) and adapts ELI's delivery to match. Crisis-aware: an STT-robust self-harm detector steers ELI into a supportive response when needed.

30 · Self-improvement & LoRA

ELI can study its own failures and run patch cycles (`SELF_ANALYZE` / `SELF_IMPROVE` / `SELF_PATCH`), keep a log, run and review its own test suite, and *write new tests* for its own functions (sandbox-verified). It can also **fine-tune itself with LoRA**: a preflight checks the modules, base model and reviewed data; the training DAG runs preflight → build → train → eval. You dry-run it from chat; the real training runs as a scheduled overnight task so it never blocks you. Standing nightly jobs (engine-eval + test report, test generation) keep it honest over time.

Part V — Configuration & operations

31 · Settings, in full

The Settings tab is where you tune ELI. The main areas:

- **Model & hardware.** Which GGUF model, and how it loads. ELI recommends GPU layers, batch and context for your machine and fits within your measured free VRAM by reducing, in order, GPU layers → batch → context (context last). Your chosen context is the anchor; explicit settings win, and ELI reduces only to make things fit.
- **Voice.** Input/output on/off, the wake word, the TTS voice, and your learned voice profile.
- **Network.** The Net toggle, and — for power users — Full Control, which lifts the offline barrier by design (see §32).
- **Persona & behaviour.** How ELI presents itself and how proactive it is.
- **Connectivity.** Audio input/output devices, the phone dashboard / Connect tab, and smart-home devices, rooms and scenes.

32 · Privacy, network & security

> **Offline by default, enforced at the socket**
>
> ELI does all inference and memory on your machine. Reaching the internet requires the **Net** toggle — enforced, not requested: a process-wide socket guard refuses non-loopback connections while Net is off, installed at server startup before anything can try to phone home. Every networked action (web search, news, weather) also routes through a single gate that fails closed. Turn Net on for a task, off after.

- **Full Control** is an explicit power-user override that lifts the offline barrier. If web actions run with Net off, check whether Full Control is on — that's by design.
- **Gated shell.** Shell commands and generated scripts pass a risk gate, an approval step, and a safety wrapper before anything executes.
- **Tamper-evident audit.** Every API action is recorded into a hash-chained ledger (the Audit surface) — who did what, with what outcome.
- **RBAC on the server.** The web API distinguishes read-only viewers from members; acting endpoints (chat, execute) are 403'd for viewers.
- **Your data never leaves.** Conversations, memories and files are never sent anywhere for the model to think. No telemetry phone-home.

33 · Models & hardware

ELI is model- and hardware-agnostic. Drop any chat/instruct `.gguf` into `models/` and it's found and fitted; vision models need their matching `mmproj` GGUF alongside. On boot ELI measures free VRAM (after vision and the embedder load) and sizes the main model to fit. It supports modern reasoning models (Qwen3, DeepSeek-R1, Mixtral): it strips `<think>` traces from output, prefills a no-think hint on utility calls, and detects the embedded chat template from the GGUF. You never hand-configure any of this unless you want to — see *Commands & Installers* for the model-download commands.

34 · Troubleshooting

Symptom	Fix
"Virtual environment not found"	You launched before installing. Run the setup first (`./ELI_Setup.sh`).
ELI won't answer / has no model	Add a GGUF model to `models/` , run `./RUN_ELI.sh --with-github-assets` , or `python -m eli.core.model_download --auto` .
Install fails on GPU bits	Reinstall CPU-only: `./INSTALL_ELI.sh --cpu-only` (or `install.bat /cpu`).
Phone can't use its microphone	Start the server with `--https` and accept the one-time self-signed-cert warning on the phone.
The Connect-a-phone QR won't render	Needs the `segno` package; bundled from v2.0.18 onward. On older installs: `pip install segno` .
It reached the web with Net off	Confirm the Net toggle persisted, and check whether **Full Control** is on (that lifts the barrier by design).
Odd behaviour at startup	Launch with `--safe-mode` ; add `--trace` for detailed logs under `artifacts/startup/logs/` .
Voice input not working	`.venv/bin/python -m eli.tools.mic_diag` , or say "run voice diagnostics".
Ask ELI itself	"run a full runtime audit", "what are you running on", "why did that fail to route", "what imports are failing" — it introspects the real runtime.

35 · Where your data lives

Because ELI is portable and self-contained, everything stays inside its own install folder:

- `artifacts/db/` — the four SQLite databases (conversations, memories, profile, agent + coding memory).
- `artifacts/runtime/` , `artifacts/startup/logs/` — runtime state and logs.
- `config/` — your settings.
- `models/` , `tts_piper/` — the AI model and voice weights.

Back up or move the whole folder and nothing is lost. To uninstall, delete the folder (and, for an AppImage install, `~/.local/share/ELI_v2` plus any `.desktop` launchers).

ELI v2.0 — your assistant, your hardware, your data. This manual documents v2.0.18. For the quick capability tour see *What ELI Is & Can Do*; for exact commands and flags see *Commands & Installers*; to get set up see the *New User Install Guide*.